

Karmaşıklık Analizi

Veri yapıları ve Algoritmalar



Matris Çarpımı

Algoritmanın Çalışma Mantığı

- Matris çarpımında sonuç matrisindeki her eleman şu şekilde hesaplanır:

$$C[i][j] = \sum A[i][k] \times B[k][j]$$

Matris Çarpımı

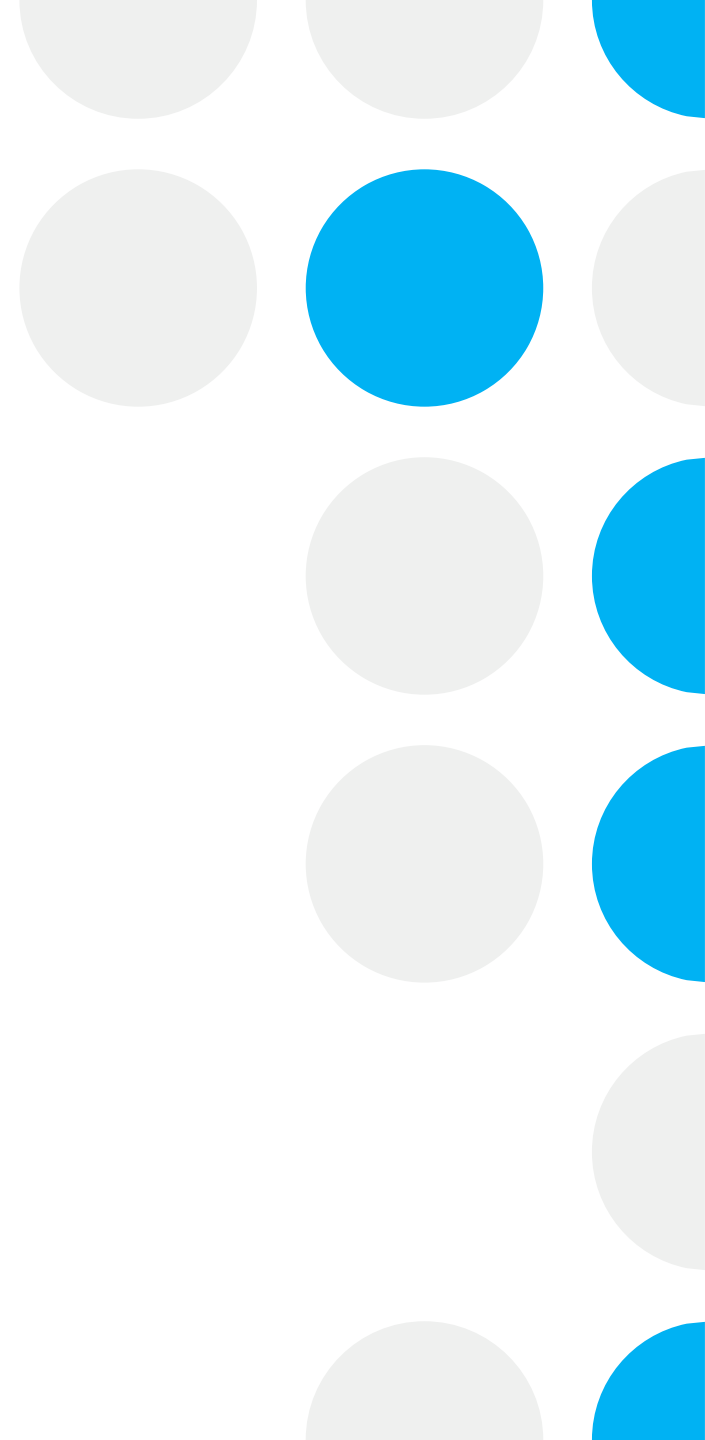
Yani:

- **A matrisinin i. satırı**
- **B matrisinin j. sütunu**

çarpılır ve elde edilen değerler toplanır.

Bu işlem kodda **üç iç içe döngü** ile yapılmaktadır:

- **i döngüsü** → **A matrisinin satırlarını gezer**
 - **j döngüsü** → **B matrisinin sütunlarını gezer**
 - **k döngüsü** → **çarpım ve toplama işlemini yapar**
-



```
int[][] sonucMatris = new int[satirA][sutunB];

for (int i = 0; i < satirA; i++) { —————→ n
    for (int j = 0; j < sutunB; j++) { —————→ n
        for (int k = 0; k < sutunA; k++) { —————→ n
            sonucMatris[i][j] += matrisA[i][k] * matrisB[k][j];
        }
    }
}
```

Matris Çarpımı

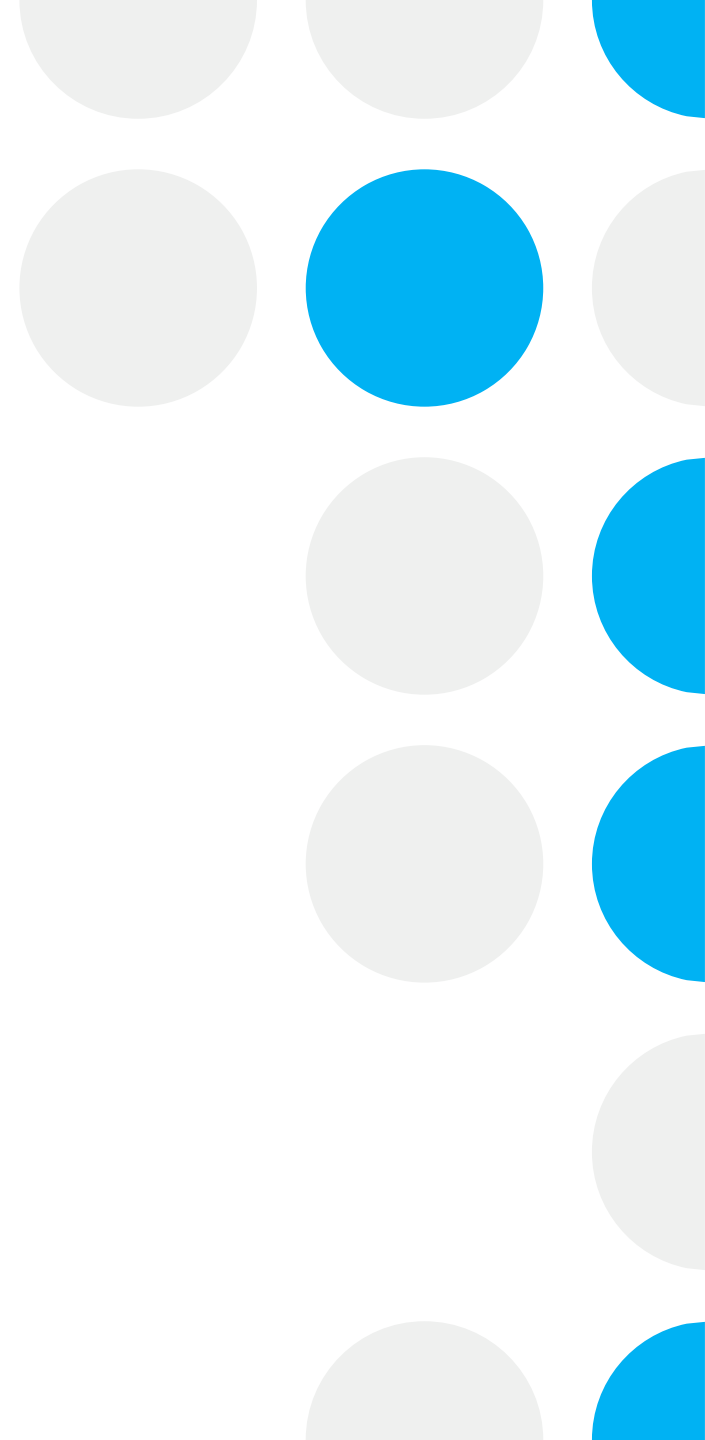
Toplam işlem sayısı:

$$n \times n \times n = n^3$$

Bu nedenle algoritmanın **zaman karmaşıklığı**

$$O(n^3)$$

olarak ifade edilir.



Matris Transpoz Alma

Orijinal matris

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Transpoz

$$\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Bir matrisin **transpozu**, satır ve sütunlarının yer değiştirmesi ile elde edilir.

- Matematiksel olarak:
- $T[j][i] = A[i][j]$
- Orijinal matrisin **satırları sütunlara** **sütunları satırlara** dönüşür.

Matris Transpoz Alma

Kodda bu işlem **iki iç içe döngü** kullanılarak gerçekleştirilir.

- **i döngüsü** → **satırları** gezer
- **j döngüsü** → **sütunları** gezer

Her eleman için:

- $\text{transpozMatris}[j][i] = \text{matris}[i][j]$
-

```
for (int i = 0; i < satir; i++) { → n
    for (int j = 0; j < sutun; j++) { → n
        transpozMatris[j][i] = matris[i][j];
    }
}
```

Matris Transpoz Alma

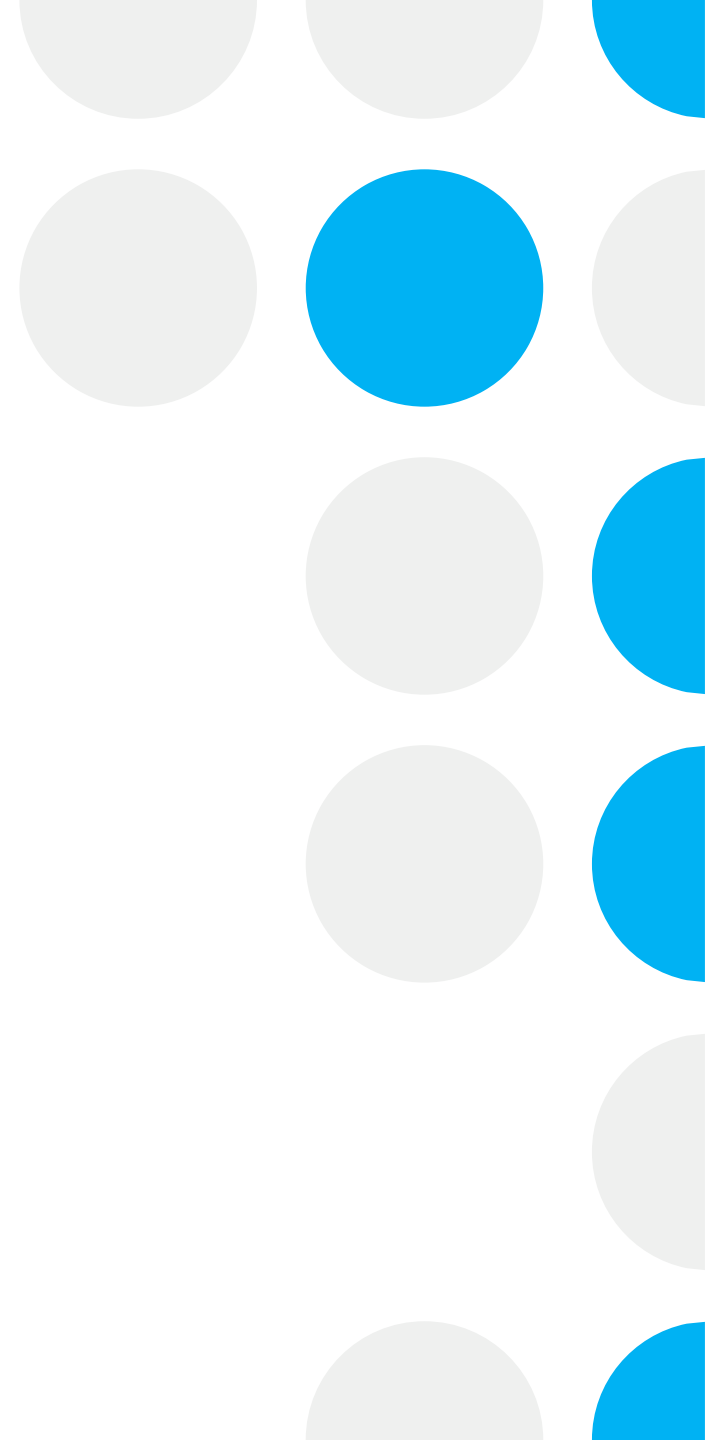
Toplam işlem sayısı:

$$n \times n = n^2$$

Bu nedenle algoritmanın **zaman karmaşıklığı**

$$O(n^2)$$

olarak ifade edilir.



2 Boyutlu Dizide Maksimum Eleman Bulma

- Algoritma şu adımlarla çalışır:
 - İlk eleman **maksimum değer** olarak kabul edilir.
 - Dizi içerisindeki tüm elemanlar **satır ve sütun döngüleri ile tek tek** gezilir.
 - Eğer mevcut eleman **maksimum değerden büyükse**, maksimum değeri güncellenir.
-

```
int satirl = dizi.length;
int sutunl = dizi[0].length;

int maksimum = dizi[0][0];

for (int i = 0; i < satirl; i++) {
    for (int j = 0; j < sutunl; j++) {
        if (dizi[i][j] > maksimum) {
            maksimum = dizi[i][j];
        }
    }
}

System.out.println("Dizideki maksimum eleman: " + maksimum);
```



2 Boyutlu Dizide Maksimum Eleman Bulma

Toplam işlem sayısı:

$$n \times n = n^2$$

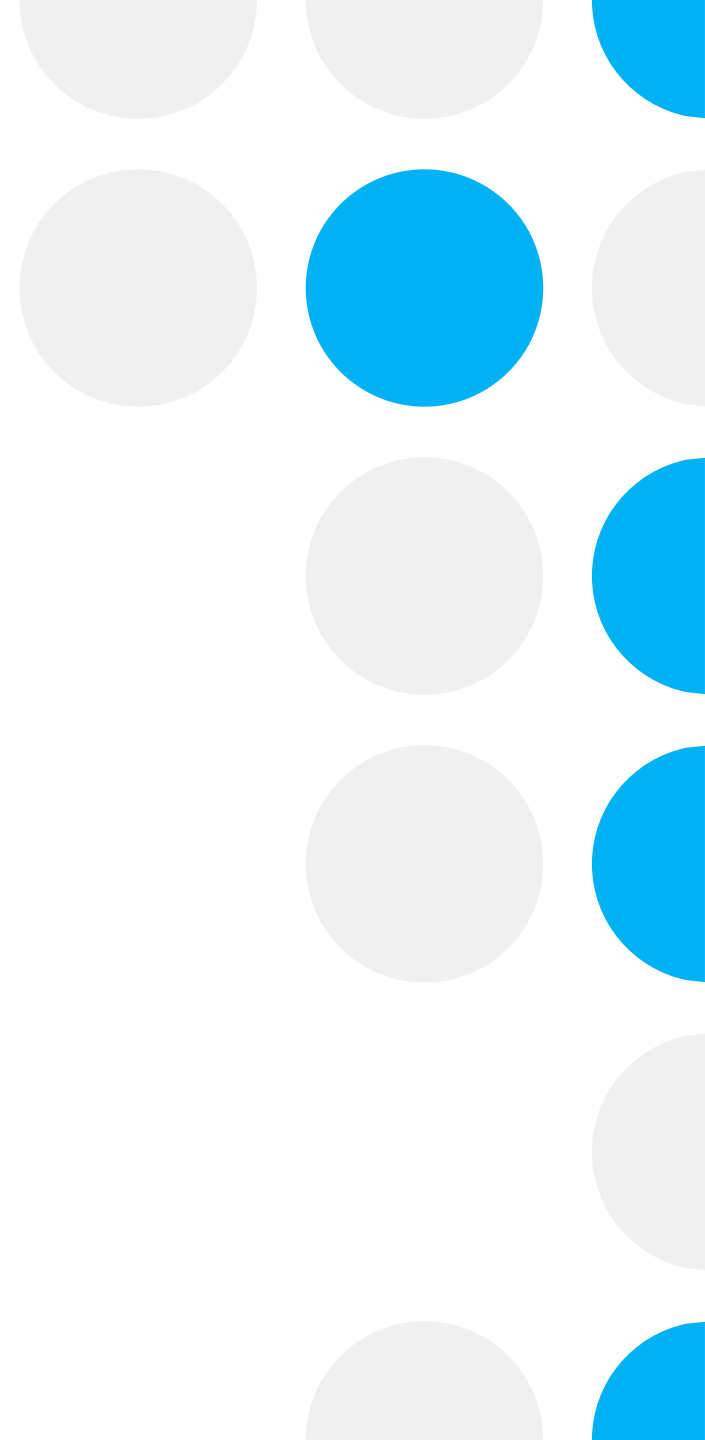
Bu nedenle algoritmanın **zaman karmaşıklığı**

$$O(n^2)$$

olarak ifade edilir.

İki Listenin Kesişimini Bulma

- Algoritma şu şekilde çalışır:
 - **Birinci liste elemanları tek tek alınır.**
 - Her eleman için **ikinci listenin tüm elemanları kontrol edilir.**
 - Eğer iki listede **aynı değer bulunursa**, bu değer kesişim listesine eklenir.
-



```
int[] listel = {1, 3, 5, 7, 9};
```

```
int[] liste2 = {2, 3, 5, 8, 9};
```

```
ArrayList<Integer> kesisim = new ArrayList<>();
```

```
for (int i = 0; i < listel.length; i++) {
```

```
    for (int j = 0; j < liste2.length; j++) {
```

```
        if (listel[i] == liste2[j]) {
```

```
            kesisim.add(listel[i]);
```

```
        }
```

```
    }
```

```
}
```

```
System.out.println("Kesişim elemanları:");
```

```
for (int sayi : kesisim) {
```

```
    System.out.print(sayi + " ");
```

```
    System.out.println("");
```

```
}
```

n

m

İki Listenin Kesişimini Bulma

Toplam karşılaştırma sayısı:

$$n \times m$$

Bu nedenle algoritmanın **zaman karmaşıklığı**

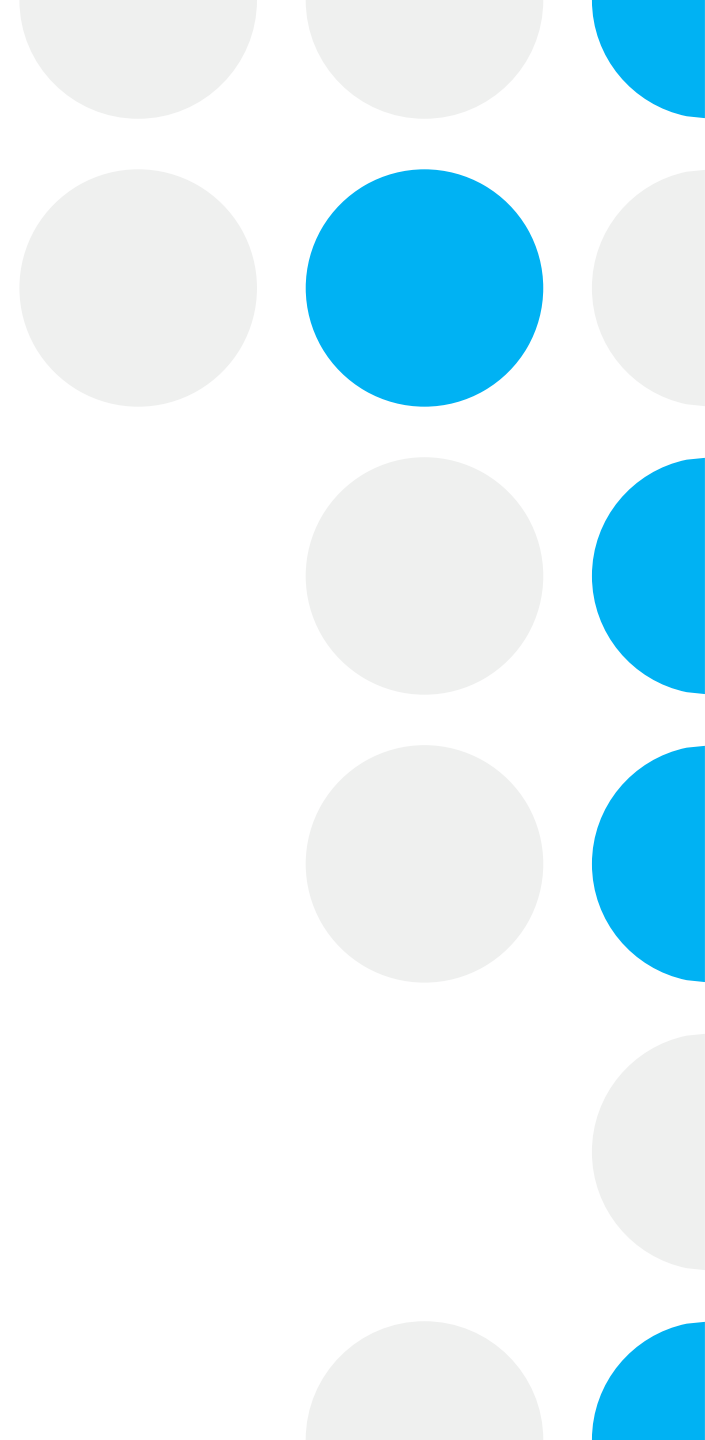
$$O(n \cdot m)$$

olarak ifade edilir.

Bir Sayının Bit Sayısını Bulma

- Bu algoritmanın amacı bir sayının **ikili sayı sisteminde kaç bit ile temsil edildiğini** bulmaktır.
 - Algoritma şu şekilde çalışır:
 - Sayı her adımda **2'ye bölünür**.
 - Her bölme işlemi bir **bit basamağını** temsil eder.
 - Sayı **0 olana kadar** işlem devam eder ve yapılan işlem sayısı bit sayısını verir.
-

```
while (sayi > 0) {  
    sayi = sayi / 2;  n/2  
    bitSayisi++;  
}
```



Her adımda sayı **ikiye bölünmektedir**.

İşlem sırası:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$$

Bu işlem şu noktaya kadar devam eder:

$$\frac{n}{2^k} = 1$$

Buradan:

$$k = \log_2 n$$

elde edilir. Bu nedenle algoritmanın **zaman karmaşıklığı**

$$O(\log n)$$

olarak ifade edilir.
